# ЗВІТ

**до лабораторної роботи №1**

**з дисципліни «Системне та мережне програмування»**

Тема: Процеси та потоки. Взаємодія між процесами.

Розподіл даних через Memory-Mapped Files

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.09.000 – Лаб1 | | |
| Змн. | Арк. | № докум. | Підпис | Дата | | | |
| Розроб. | | Зінченко М. В. | | | | Літ. | Арк. / Аркушів |
| Перевір. | | Власенко О. В. | | | | | 1 / 17 |
| Реценз. | | | | | | | |
| Н. Контр. | | | | | | ФІКТ, гр. ІПЗ-23-5 | |
| Зав.каф. | | Вакалюк Т.А. | | | | | |

## 1. Мета роботи

- Вивчити механізми взаємодії між процесами (Inter-Process Communication, IPC).
- Освоїти роботу з файлами, відображеними у пам'ять (Memory-Mapped Files).
- Навчитись організовувати одночасний доступ декількох процесів до спільних даних.
- Засвоїти принципи синхронізації потоків і процесів (mutex, semaphore, події).
- Навчитись працювати зі структурами WinAPI для роботи з файлами, пам'яттю, процесами та потоками.
- Освоїти обробку виняткових ситуацій, визначення критичних секцій та захист спільних ресурсів.

## 2. Завдання лабораторної роботи

Створити чотири взаємодіючі програми, які працюють з одним спільним файлом data.dat через Memory-Mapped File:

- Програма №3 - генератор масиву (створює data.dat, 20-30 випадкових чисел [10;100], бінарно).
- Програма №1 - консольне сортування масиву (Bubble Sort) з обробкою клавіші "пробіл".
- Програма №2 - віконна візуалізація масиву (стовпчики з символів "*", таймер SetTimer).
- Програма №4 (додаткове завдання) - паралельний сортувальник (QuickSort, за спаданням), що працює одночасно з програмою №1 над тим самим масивом.

Усі програми синхронізуються через іменований мьютекс Global\MMF_MUTEX.

## 3. Теоретичні відомості

### 3.1. Процеси і потоки в ОС Windows

Процес - це виконуваний екземпляр програми, якому операційна система виділяє власний ізольований адресний простір, набір handle-ів на відкриті ресурси (файли, mapping-об'єкти, синхронізаційні примітиви) та як мінімум один потік виконання. Потік (thread) - це одиниця планування процесора всередині процесу; кілька потоків одного процесу виконуються паралельно, але спільно використовують той самий адресний простір, купу (heap) та глобальні змінні.

Головна відмінність процес -> потік полягає саме в ізоляції пам'яті: два потоки одного процесу можуть напряму читати й писати одні й ті самі змінні, тоді як два різних процеси фізично не бачать пам'ять один одного - навіть якщо обидва працюють з однаковими адресами вказівників, ці адреси відносяться до різних фізичних сторінок пам'яті. Саме тому для обміну даними між процесами (на відміну від потоків) потрібен окремий механізм - IPC.

У даній роботі кожна з чотирьох програм (№1-№4) - це окремий процес з власним ізольованим адресним простором. Спільний доступ до масиву даних вони отримують не напряму, а через Memory-Mapped File: операційна система відображає одні й ті самі фізичні сторінки пам'яті (підкріплені файлом data.dat) у віртуальний адресний простір кожного з цих процесів окремо.

### 3.2. Методи взаємодії процесів (IPC)

Windows надає кілька механізмів міжпроцесної взаємодії (Inter-Process Communication):

- Канали (pipes) - потокова передача байтів через анонімний (Anonymous Pipe) або іменований (Named Pipe) канал; підходить для послідовного обміну повідомленнями між батьківським і дочірнім процесом.
- Сокети (sockets) - універсальний механізм, що працює як локально, так і по мережі; має найбільші накладні витрати через мережевий стек протоколів.
- Черги повідомлень (Message Queues) - асинхронна передача структурованих повідомлень через чергу ядра.
- Спільна пам'ять (Shared Memory) - область пам'яті, доступна кільком процесам одночасно; у Windows реалізується саме через механізм Memory-Mapped Files.
- Memory-Mapped Files (MMF) - найшвидший спосіб обміну даними, оскільки після початкового налаштування (виклики CreateFileMapping/MapViewOfFile) подальший обмін відбувається без жодних системних викликів: процес просто читає й пише байти за вказівником, як у звичайний масив у оперативній пам'яті.

MMF обрано найшвидшим саме тому, що канали, сокети та черги вимагають копіювання даних через буфери ядра (kernel buffer) при кожній операції передачі, тоді як MMF дає прямий доступ до спільних фізичних сторінок пам'яті без проміжного копіювання.

### 3.3. Memory-Mapped File (MMF)

Механізм MMF працює у три етапи. Спочатку відкривається звичайний файл на диску (CreateFile) - він виступає "підкладкою" (backing store) для даних. Далі створюється об'єкт відображення файлу (CreateFileMapping) - системний об'єкт ядра, який описує, яку частину файлу і з якими правами доступу дозволено відображати в пам'ять. Нарешті викликом MapViewOfFile ця область файлу проєктується у віртуальний адресний простір конкретного процесу, і функція повертає звичайний вказівник (void*), через який можна читати й писати дані так само, як у масив.

Ключова властивість MMF, яка й уможливлює IPC: якщо кілька різних процесів викликають CreateFileMapping з одним і тим самим іменем об'єкта (у даній роботі - "Global\\MMF_DATA_ARRAY"), система не створює новий mapping-об'єкт, а повертає handle на вже існуючий. Відповідно, коли кожен із цих процесів після цього викликає MapViewOfFile, операційна система проєктує в їхні окремі адресні простори одні й ті самі фізичні сторінки

пам'яті. Зміна байта, зробена одним процесом, миттєво видима іншому - без будь-якого копіювання чи додаткових системних викликів, оскільки фізично це та сама область оперативної пам'яті.

У цій роботі файл data.dat трактується як масив 32-бітних цілих чисел: вказівник, отриманий від MapViewOfFile, приводиться до типу int*, і подальша робота з масивом (arr[i]) - це вже звичайні звернення до пам'яті, без жодного файлового вводу-виводу. Функція FlushViewOfFile використовується для примусового скидання змінених сторінок на диск, коли потрібна гарантія збереження даних, а не лише їх видимість іншим процесам (яка забезпечується автоматично самим механізмом відображення).

### 3.4. Синхронізація процесів

Оскільки кілька процесів одночасно мапують один і той самий масив, без синхронізації виникає класична проблема гонки даних (data race): один процес може читати елемент масиву в момент, коли інший процес лише наполовину записав у нього нове значення, або два процеси можуть одночасно виконати перестановку елементів, і результат буде непередбачуваним.

Windows надає для цього об'єкти ядра синхронізації - Mutex (взаємне виключення, дозволяє лише одному потоку/процесу володіти ним одночасно), Semaphore (лічильник, що дозволяє обмежену кількість одночасних власників) та Event (прапорець сигналізації для повідомлення про подію). У даній роботі обрано саме Mutex, оскільки задача вимагає класичного взаємного виключення: доступ до масиву повинен мати рівно один процес одночасно, незалежно від того, скільки процесів очікують.

Захоплення мьютекса виконується через WaitForSingleObject(hMutex, timeout): виклик блокує потік, доки мьютекс не звільниться, після чого атомарно передає володіння викликаючому потоку. WaitForMultipleObjects застосовується, коли потрібно очікувати одночасно кілька об'єктів синхронізації (в даному проєкті не використовується, оскільки достатньо одного мьютекса). Звільнення відбувається викликом ReleaseMutex - обов'язково тим самим потоком, який мьютекс захопив.

Іменований мьютекс Global\MMF_MUTEX використовується всіма чотирма програмами: оскільки ім'я однакове, CreateMutexW у кожній програмі повертає handle на той самий kernel-об'єкт, і всі чотири процеси синхронізуються між собою, попри те що це фізично різні, незалежно запущені процеси.

### 3.5. Обробка виняткових ситуацій

Для кожної з чотирьох програм визначено такі потенційні точки відмови: відсутність файлу data.dat (якщо програму №3 не запущено першою), пошкоджений або порожній файл (розмір 0 або не кратний sizeof(int)), відмова створення mapping-об'єкта чи відображення у пам'ять (наприклад, через нестачу віртуальної адресної пам'яті), недоступність мьютекса протягом розумного часу очікування, а також вихід одного з процесів під час утримання ним мьютекса ("покинутий" мьютекс, WAIT_ABANDONED).

Критичними секціями визнано всі ділянки коду, де відбувається безпосередній доступ до масиву в MMF: один прохід сортування (програма №1), одне розбиття QuickSort (програма №4), копіювання масиву для відображення (програма №2) та початковий запис масиву (програма №3). Ризик взаємного блокування (deadlock) у даному проєкті мінімізовано тим, що використовується лише один мьютекс і жодна з програм не намагається захопити його рекурсивно або утримувати два різні об'єкти синхронізації одночасно у зустрічному порядку - класична причина deadlock. Додатковим захистом слугує скінченний таймаут у WaitForSingleObject замість INFINITE: якщо мьютекс не звільняється надто довго, це виявляється як попередження, а не як безумовне зависання програми.

## 4. Опис програм

### 4.3. Програма №3 - «Генератор масиву»

**Призначення програми:**

Створює або перезаписує бінарний файл data.dat, заповнюючи його 20-30 випадковими 32-бітними цілими числами у діапазоні [10;100]. Запускається першою, автономно від інших програм.

**Глобальні змінні:**

Власних глобальних змінних немає (окрім спільних констант з common.h, див. Додаток).

**Структура програми (основні кроки):**

- Ініціалізація генератора випадкових чисел (srand).
- Визначення кількості елементів масиву (20-30).
- Генерація масиву випадкових значень [10;100].
- Захоплення мьютекса Global\MMF_MUTEX.
- Створення файлу data.dat (CREATE_ALWAYS) та бінарний запис масиву (WriteFile).
- Звільнення мьютекса та handle-ів.

**Методи та прийоми, використані при вирішенні задачі:**

Дані записуються у файл у бінарному вигляді (WriteFile напряму з масиву int[]), а не у вигляді тексту - це дозволяє програмам №1, №2 та №4 інтерпретувати вміст файлу безпосередньо як масив int* після відображення в пам'ять, без окремого етапу парсингу. Режим CREATE_ALWAYS обрано свідомо: кожен запуск генератора повинен давати новий, повністю незалежний набір даних, незалежно від того, чи існував файл раніше. Хоча програма №3 працює автономно до старту решти процесів, критична секція запису все одно захищена мьютексом - це узгоджується з вимогою методички про захист усіх операцій запису у файл, а також рятує від пошкодження даних у сценарії, коли попередній запуск однієї з програм ще не встиг звільнити ресурси.

## 4.1. Програма №1 - «Сортування даних»

**Призначення програми:**

Консольна програма (на основі "Hello Windows"), яка за натисканням клавіші "пробіл" відображає data.dat у пам'ять через MMF і виконує сортування методом Bubble Sort з затримкою 1 секунда після кожної ітерації, зберігаючи зміни безпосередньо у MMF.

**Глобальні змінні:**

Власних глобальних змінних немає (окрім спільних констант з common.h, див. Додаток).

**Структура програми (основні кроки):**

- Очікування натискання клавіші (_kbhit/_getch) у неблокуючому циклі.
- За натисканням пробілу: відкриття мьютекса та підключення до MMF (OpenExistingDataMmf).
- Цикл ітерацій Bubble Sort: захоплення мьютекса -> один прохід сортування -> FlushViewOfFile -> звільнення мьютекса -> Sleep(1000).
- Повторення до завершення сортування (відсутність перестановок).
- Виведення "Робота завершена" та звільнення ресурсів.

**Методи та прийоми, використані при вирішенні задачі:**

Обрано алгоритм Bubble Sort саме через його покроковість: на відміну від рекурсивних алгоритмів (QuickSort, MergeSort), один "прохід" бульбашкового сортування - природна, однакова за розміром одиниця роботи, між якою зручно вставити паузу в 1 секунду й дати можливість спостерігати проміжний стан масиву через програму №2. Ключовий прийом організації критичної секції - її мінімізація: мьютекс утримується лише на час одного проходу масиву й виклику FlushViewOfFile, а секундна затримка (Sleep) винесена за межі захопленого мьютекса. Якби пауза виконувалась усередині критичної секції, інші процеси (програма №2 -

для читання, програма №4 - для запису) не змогли б звернутися до масиву цілу секунду на кожній ітерації, що зробило б паралельну демонстрацію конкуренції процесів (додаткове завдання) практично непомітною. Для стійкості до зависань застосовано скінченний таймаут очікування мьютекса (5000 мс) замість INFINITE, що дозволяє виявити потенційний deadlock, а не заблокувати програму назавжди.

### 4.2. Програма №2 - «Візуалізація даних»

**Призначення програми:**

Віконна WinAPI-програма, яка відображає data.dat у пам'ять і кожні 0.5 секунди (SetTimer) перемальовує вікно, виводячи кожен елемент масиву як рядок із символів "*", кількість яких дорівнює значенню елемента.

**Глобальні змінні:**

| Ім'я | Тип | Призначення |
|---|---|---|
| g_ctx | MmfContext | Контекст MMF (handle файлу, mapping, вказівник на масив, мьютекс) - глобальна, бо WndProc не приймає власних параметрів користувача |
| g_mmfReady | bool | Прапорець готовності MMF до читання (встановлюється після успішного WM_CREATE) |

**Структура програми (основні кроки):**

- WM_CREATE: підключення до мьютекса та MMF, запуск таймера SetTimer(500 мс).
- WM_TIMER: запит на перемальовку вікна через InvalidateRect.
- WM_PAINT: захоплення мьютекса -> копіювання масиву в локальний буфер -> звільнення мьютекса -> малювання рядків "*" через TextOutW (поза критичною секцією).
- WM_DESTROY: зупинка таймера, звільнення ресурсів MMF, завершення програми.

**Методи та прийоми, використані при вирішенні задачі:**

Застосовано стандартний для WinAPI прийом розділення відповідальності між WM_TIMER та WM_PAINT: таймер лише позначає вікно як "застаріле" (InvalidateRect), а фактичне малювання Windows виконує самостійно через WM_PAINT у зручний для системи момент. Це узгоджується з вимогою методички "при кожному таймерному повідомленні очищати вікно й виводити масив", але без прямого виклику графічних функцій із обробника таймера, що є поганою практикою в WinAPI. Другий важливий прийом - копіювання масиву з MMF у

локальний буфер під захопленим мьютексом, а сам рендеринг (TextOutW для кожного рядка) виконується вже поза критичною секцією: операції виведення тексту у вікно можуть зайняти помітний час, і немає сенсу тримати інші процеси заблокованими на весь цей час. Використання глобальних змінних (g_ctx, g_mmfReady) - вимушений прийом, оскільки віконна процедура WndProc у стандартному WinAPI не отримує довільних користувацьких даних без додаткового підключення через SetWindowLongPtr(GWLP_USERDATA), що для навчальної задачі є надлишковим ускладненням.

### 4.4. Програма №4 - «Паралельний сортувальник (QuickSort, спадання) - додаткове завдання»

**Призначення програми:**

Консольна програма, яка працює паралельно з програмою №1 і сортує той самий масив методом QuickSort у зворотному напрямку (за спаданням), демонструючи конкуренцію двох процесів за спільний ресурс під захистом мьютекса.

**Глобальні змінні:**

Власних глобальних змінних немає (окрім спільних констант з common.h, див. Додаток).

**Структура програми (основні кроки):**

- Очікування натискання клавіші "пробіл" (аналогічно програмі №1).
- Підключення до мьютекса та MMF.
- Рекурсивний QuickSort: кожен крок розбиття (partition) виконується в окремій критичній секції (Lock -> розбиття -> Flush -> Unlock).
- Штучна затримка 300 мс поза критичною секцією між кроками - для наочної демонстрації конкуренції з програмою №1.
- Виведення "Робота завершена" та звільнення ресурсів.

**Методи та прийоми, використані при вирішенні задачі:**

Головний прийом, застосований у цій програмі, - інверсія умови порівняння у функції розбиття Лом'юто (Lomuto partition): замість стандартного "<=" використано ">=", що змінює результуючий порядок сортування із зростання на спадання без потреби писати окремий алгоритм. Другий важливий прийом - дроблення критичної секції на рівні одного кроку розбиття, а не всього рекурсивного виклику QuickSort. Якби мьютекс захоплювався один раз на початку сортування й утримувався до повного завершення, програма №4 монопольно заблокувала б масив на весь час своєї роботи, і одночасне (конкуруюче) сортування з програмою №1 стало б неможливим - весь сенс додаткового завдання (продемонструвати, що два процеси одночасно намагаються сортувати один масив) було б втрачено. Додана штучна

затримка в 300 мс між кроками розбиття вирівнює швидкість QuickSort із секундними ітераціями Bubble Sort у програмі №1, щоб конкуренцію процесів можна було реально спостерігати у вікні візуалізації, а не побачити миттєво завершений результат.

## 5. Опис використаних WinAPI-функцій

Розбір виконано за шаблоном методичних вказівок: призначення, основні параметри, повертане значення, особливості.

| **Функція** | **Призначення** | **Основні параметри** | **Особливості / повернене значення** |
|---|---|---|---|
| CreateFileW | Відкриває існуючий або створює новий файл | ім'я файлу; режим доступу (GENERIC_READ/WRITE); режим спільного доступу; спосіб створення (OPEN_EXISTING/CREATE_ALWAYS) | Повертає HANDLE або INVALID_HANDLE_VALUE при помилці; код помилки - через GetLastError (напр. ERROR_FILE_NOT_FOUND, ERROR_ACCESS_DENIED) |
| CreateFileMappingW | Створює об'єкт відображення файлу в пам'ять (file mapping) | handle файлу; атрибути захисту; PAGE_READWRITE; розмір (0,0 = розмір файлу); ім'я об'єкта | Повертає HANDLE mapping-об'єкта або nullptr; кілька процесів з однаковим ім'ям отримують спільний mapping |
| MapViewOfFile | Відображає (мапить) файл у адресний простір процесу | handle mapping-об'єкта; права доступу (FILE_MAP_ALL_ACCESS); зміщення; розмір відображення (0 = весь файл) | Повертає вказівник (void*) на початок відображеної області або nullptr при помилці |
| FlushViewOfFile | Примусово скидає змінені сторінки з пам'яті на диск | вказівник на область; розмір (0 = вся область) | Гарантує збереження змін у файлі без очікування системного кешування |
| UnmapViewOfFile | Відміняє відображення файлу з пам'яті процесу | вказівник, отриманий від MapViewOfFile | Має викликатись перед CloseHandle для mapping-об'єкта |
| CloseHandle | Закриває | handle об'єкта | Об'єкт видаляється системою, коли |

| Функція | Призначення | Основні параметри | Особливості / повернене значення |
|---|---|---|---|
| | будь-який kernel-об'єкт (файл, mapping, мьютекс тощо) | | останній handle на нього закрито |
| CreateMutexW | Створює або відкриває іменований мьютекс | атрибути захисту; початковий власник (FALSE); ім'я об'єкта (Global\MMF_MUTEX) | Якщо об'єкт з таким ім'ям вже існує - повертає handle на нього; GetLastError()==ERROR_ALREADY_EXISTS не є помилкою |
| WaitForSingleObject | Очікує звільнення (сигналізації) об'єкта синхронізації | handle об'єкта; таймаут у мс (або INFINITE) | Повертає WAIT_OBJECT_0 (успіх), WAIT_TIMEOUT (таймаут - ознака можливого deadlock) або WAIT_ABANDONED (попередній власник завершився аварійно) |
| ReleaseMutex | Звільняє раніше захоплений мьютекс | handle мьютекса | Повинен викликатись тим самим потоком, який захопив мьютекс |
| SetTimer | Створює таймер вікна | handle вікна; ідентифікатор таймера; інтервал у мс; функція зворотного виклику (nullptr - використовується WM_TIMER) | Генерує повідомлення WM_TIMER із заданим інтервалом |
| InvalidateRect | Позначає область вікна такою, що потребує перемалювання | handle вікна; область (nullptr - усе вікно); прапорець очищення фону | Спричиняє надсилання повідомлення WM_PAINT |
| BeginPaint / EndPaint | Готує/завершує контекст пристрою для малювання під час обробки WM_PAINT | handle вікна; структура PAINTSTRUCT | Обов'язкова пара викликів саме в обробнику WM_PAINT |

| Функція | Призначення | Основні параметри | Особливості / повернене значення |
|---|---|---|---|
| TextOutW | Виводить текстовий рядок у вказаній позиції вікна | контекст пристрою; координати x,y; текст; довжина тексту | Використовується для відображення рядків із символів "*" |

**Константи, пов'язані з функцією CreateFileW (параметр dwCreationDisposition):**

- CREATE_ALWAYS - завжди створює новий файл, перезаписуючи існуючий (використано у програмі №3).
- OPEN_EXISTING - відкриває файл, лише якщо він вже існує; помилка ERROR_FILE_NOT_FOUND, якщо файлу немає (використано у програмах №1, №2, №4).
- CREATE_NEW - створює новий файл; помилка, якщо файл вже існує (у проекті не використовується).
- OPEN_ALWAYS - відкриває файл, або створює, якщо не існує (у проекті не використовується).
- TRUNCATE_EXISTING - відкриває і обрізає існуючий файл до нульової довжини (у проекті не використовується).

## 6. Синхронізація процесів. Критичні секції

Усі чотири програми використовують один іменований мьютекс Global\MMF_MUTEX. Це гарантує, що в будь-який момент часу лише один процес модифікує або читає масив у MMF.

**Виявлені критичні секції:**

| № | Програма | Критична секція | Що захищається |
|---|---|---|---|
| 1 | №3 | Запис масиву у щойно створений файл | Цілісність даних під час першого запису |
| 2 | №1 | Один прохід Bubble Sort (порівняння + перестановка сусідніх елементів) | Узгодженість масиву під час сортування |
| 3 | №2 | Копіювання масиву з MMF у локальний буфер перед малюванням | Читання без "розриву" даних під час одночасного запису |
| 4 | №4 | Один крок розбиття (partition) QuickSort | Узгодженість масиву при конкуруючому сортуванні |

| № | Програма | Критична секція | Що захищається |
|---|---|---|---|
| | | | з програмою №1 |

Межі кожної критичної секції обрано мінімально можливими - лише той код, що безпосередньо звертається до спільного масиву в MMF (порівняння, перестановка, копіювання). Операції, що не потребують ексклюзивного доступу до масиву - очікування клавіші, затримка Sleep, малювання тексту у вікні, генерація випадкових чисел - навмисно винесені за межі блокування. Якби критичну секцію розширити й включити в неї, наприклад, паузу Sleep(1000) у програмі №1, інші процеси (програма №2 для читання, програма №4 для запису) не змогли б звернутися до масиву протягом усієї секунди на кожній ітерації - це різко уповільнило б і спотворило б спостережувану паралельну роботу програм, хоча формально коректність даних не постраждала б. Якби ж, навпаки, критичну секцію звузити (наприклад, прибрати захоплення мьютекса на етапі FlushViewOfFile), з'явився б реальний ризик гонки даних: інший процес міг би прочитати чи змінити масив саме в момент, коли поточні зміни ще не зафіксовано.

## 7. Обробка виняткових ситуацій

У програмах реалізовано перевірку наступних сценаріїв (без аварійного завершення процесу):

- Файл data.dat не існує (ERROR_FILE_NOT_FOUND) - виводиться повідомлення з проханням спочатку запустити програму №3.
- Розмір файлу дорівнює нулю або пошкоджений - перевірка через GetFileSizeEx перед створенням mapping.
- Помилка створення file-mapping об'єкта (CreateFileMappingW повертає nullptr).
- Помилка відображення файлу у пам'ять (MapViewOfFile повертає nullptr).
- Недоступність мьютекса / таймаут очікування (WAIT_TIMEOUT) - можлива ознака взаємного блокування (deadlock).
- "Покинутий" мьютекс (WAIT_ABANDONED) - попередній власник завершився аварійно, не звільнивши ресурс.

У кожному випадку ресурси звільняються через ReleaseMmfContext, а програма продовжує роботу в очікуванні наступної дії користувача, замість аварійного завершення.

## 8. Скріншоти роботи програм

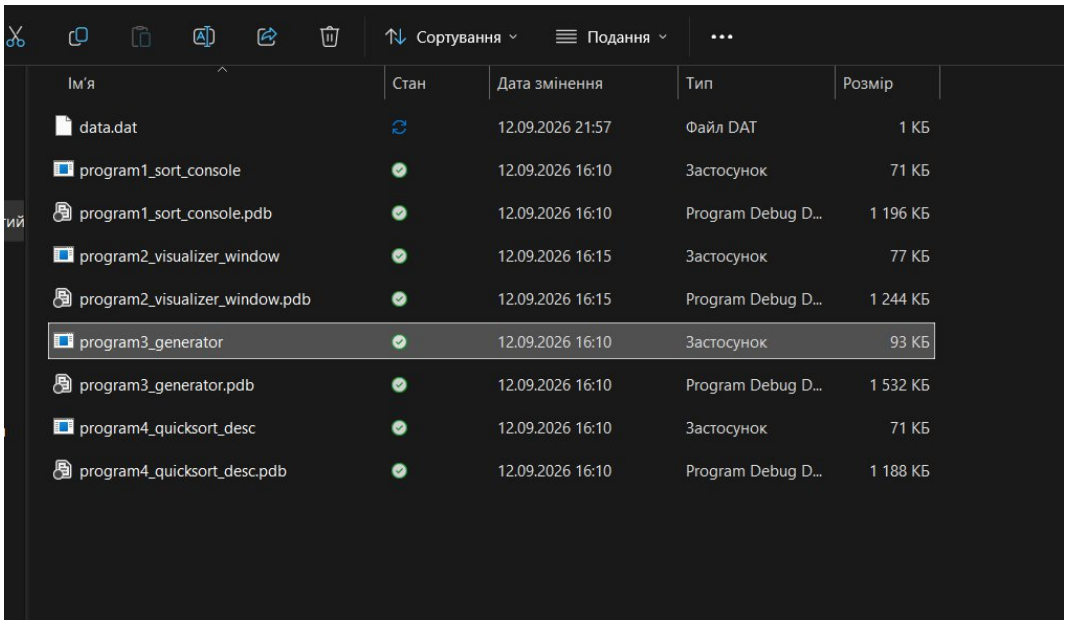



**Рис 1. Робота програми №3 (генератор масиву)(надто швидко виконується)**




**Рис 2. Робота програми №2 (візуалізація) до початку сортування**

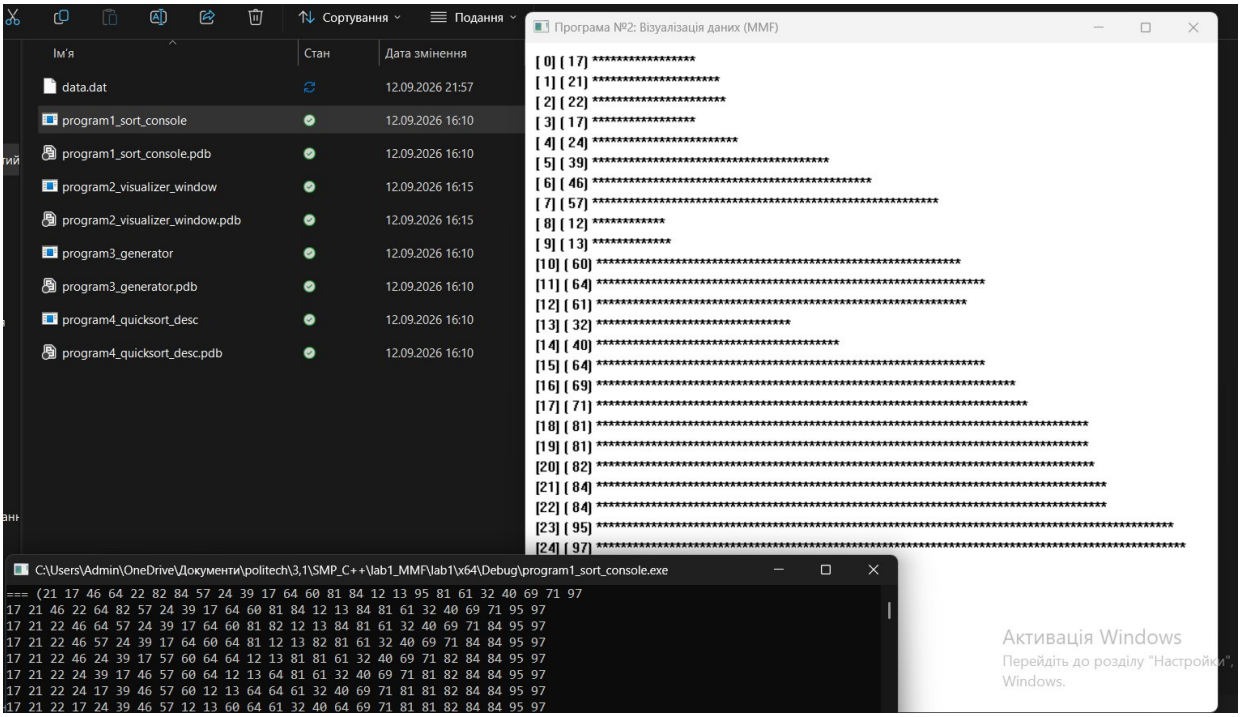



**Рис 3. Робота програм №1 та №2 одночасно (процес сортування)**

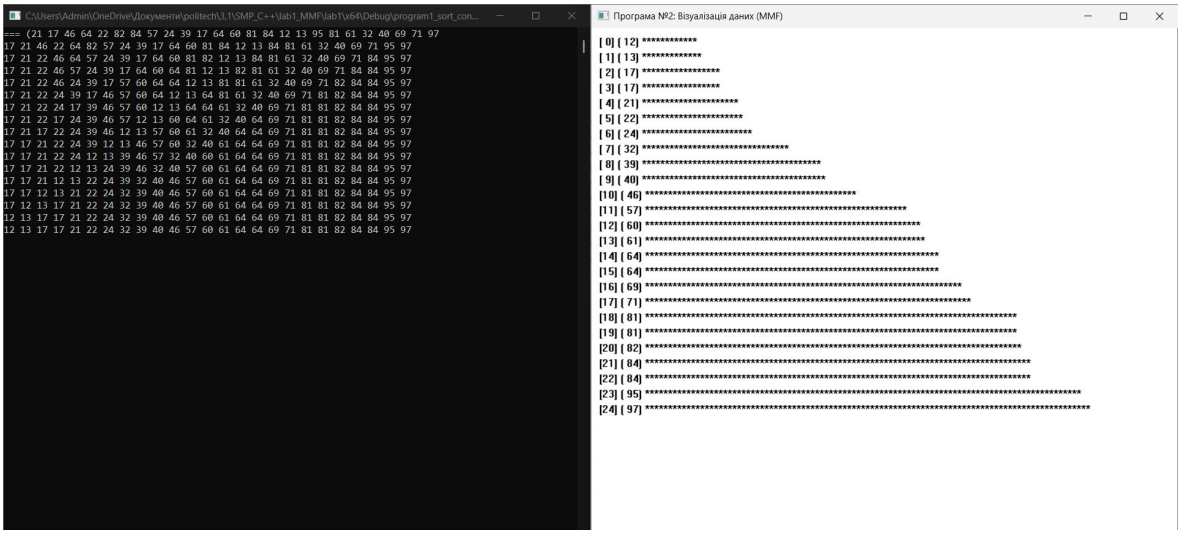
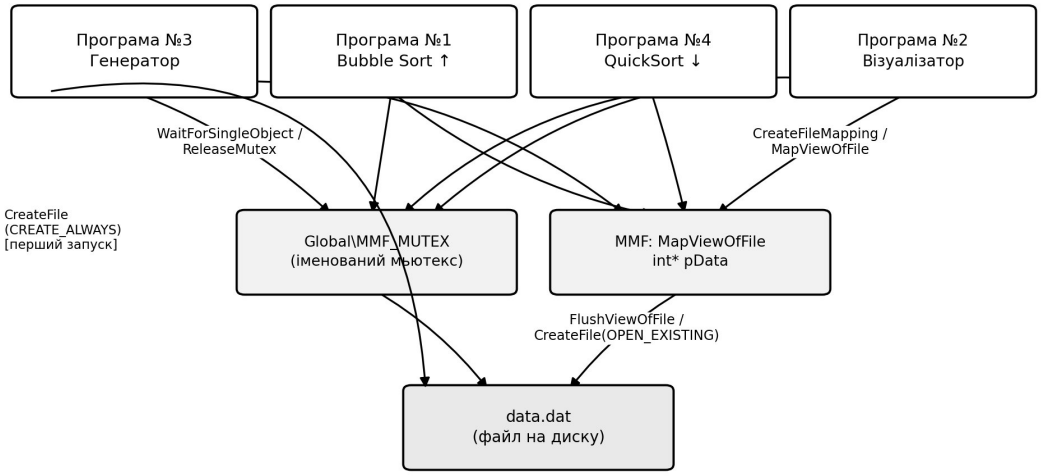



**Рис 4. Робота програми №1 та №2 після завершення сортування**




**Рис 5. Одночасна робота програм №1 і №4 (додаткове завдання)**

## 9. Діаграма взаємодії процесів




*Усі 4 процеси відображають один і той самий файл data.dat у власний адресний простір через MapViewOfFile; доступ до масиву синхронізується єдиним мьютексом Global\MMF_MUTEX.*

*Діаграма взаємодії процесів №1-№4 через MMF та мьютекс Global\MMF_MUTEX*

## 10. Аналіз

### 10.1. Артефакти при відсутності мьютекса

Якщо тимчасово прибрати захоплення мьютекса (LockMmf/UnlockMmf) навколо критичних секцій і запустити одночасно програму №1 (сортування за зростанням) та програму №4 (сортування за спаданням) над одним і тим самим масивом, виникає класична гонка даних. Обидва процеси незалежно читають і переставляють елементи одного масиву, тому можливі такі артефакти: "стрибки" значень, коли один процес переставив пару елементів, а другий у той самий момент читав застаріле значення й переставив уже неправильну пару; втрата змінених даних, коли запис одного процесу перезаписується запуском FlushViewOfFile іншого процесу зі своєю (застарілою) версією масиву в локальному кеші сторінок; повне порушення інваріанта сортування - масив може ніколи не стабілізуватися в жодному впорядкованому вигляді, оскільки два алгоритми з протилежними цілями постійно "перетягують" одні й ті самі елементи в різні боки. У вікні візуалізації (програма №2) це виглядає як хаотичне, зациклене миготіння стовпчиків без збіжності до відсортованого стану.

### 10.2. Поведінка при неправильному завершенні однієї з програм

Якщо примусово завершити процес (наприклад, через Диспетчер задач) саме в момент, коли він утримує мьютекс усередині критичної секції, операційна система автоматично звільняє всі kernel-об'єкти цього процесу, включно з мьютексом, і переводить його у стан WAIT_ABANDONED. Наступний процес, що очікував цей мьютекс через WaitForSingleObject, все одно отримує володіння (виклик повертає WAIT_ABANDONED замість WAIT_OBJECT_0), але дані, які захищав цей мьютекс, можуть залишитися в неузгодженому проміжному стані - наприклад, масив у MMF міг бути записаний лише частково (одна перестановка виконана, а суміжний запис FlushViewOfFile не встиг завершитись). У реалізованому коді ця ситуація не призводить до аварійного завершення інших програм: функція LockMmf явно перевіряє код WAIT_ABANDONED, виводить попередження про можливу неузгодженість даних і дозволяє процесу продовжити роботу, вважаючи, що мьютекс успішно захоплено. Якщо ж аварійно завершується програма №2 (візуалізатор), це взагалі не впливає на решту процесів, оскільки вона лише читає дані й не тримає ресурси, потрібні іншим; єдиний наслідок - зникнення вікна спостереження за станом масиву.

## 11. Висновки

У ході виконання лабораторної роботи було освоєно механізм Memory-Mapped Files як найшвидший спосіб організації спільного доступу кількох процесів до одних і тих самих даних без явного копіювання через буфери ядра. Реалізовано чотири взаємодіючі програми (генератор

даних, два незалежних алгоритми сортування - Bubble Sort за зростанням і QuickSort за спаданням, та WinAPI-візуалізатор), які одночасно звертаються до спільного масиву у файлі data.dat через відображення в пам'ять.

Практично підтверджено, що синхронізація доступу до спільних даних є обов'язковою умовою коректної роботи кількох процесів: іменований мьютекс Global\MMF_MUTEX дозволив уникнути гонки даних між процесами, що одночасно читають і модифікують один масив, а мінімізація меж критичних секцій (захоплення мьютекса лише на час безпосереднього звернення до масиву, а не на весь час роботи алгоритму) виявилась ключовою для того, щоб паралельна робота процесів залишалась і коректною, і спостережуваною одночасно.

Окремо відпрацьовано підхід до обробки виняткових ситуацій без аварійного завершення програм: перевірку існування та цілісності файлу перед відображенням у пам'ять, коректне звільнення ресурсів у будь-якому сценарії завершення, а також обробку граничних випадків роботи з мьютексом - таймауту очікування (можлива ознака deadlock) та "покинутого" мьютекса після аварійного завершення власника.